Stock History API
Neil Singh

[1]Python Software Foundation, “Datetime — Basic Date and Time Types — Python 3.7.2 Documentation,” *Python.org*. https://docs.python.org/3/library/datetime.html
[2]“matplotlib.pyplot.plot — Matplotlib 3.4.1 documentation,” *matplotlib.org*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html
[3]R. Aroussi, “yfinance: Yahoo! Finance market data downloader,” *PyPI*, 2023. https://pypi.org/project/yfinance/

For my final python coding project, I decided to choose a topic that I'm both passionate about, and interact with on a daily basis. Using data from Yahoo Finance, I was able to create a program capable of providing a stock trend graph, for any date in the stock's history. It is also able to provide the specific stock price at any day in the stocks history.

The program is organized such that a series of functions are used to develop the plot produced by the code. I opted not to write this section of the program in a specific class as I didn't think it was necessary to use public and private attributes. However, in order to return the specific stock price on a certain date, I created a class called 'FindPriceAtTime' to do so. The first segment of code takes in input from the user for the specific stocks they wish to view, as well as the amount of days in the past they wish to view the stocks history for. The stocks the user wishes to view are stored in a list. Entering 'STOP' breaks the request for user input. To initialize the time bounds for the plot, two variables 'start_time' and 'end_time' were defined. Using the newly imported 'datetime' library, the current date was able to be retrieved and stored in 'end_time,' whilst the date the user wishes to view the stock's history at is subtracted from the current date and stored in 'start_time.' Using a for-loop and the finance library, the list of stocks 'tickers' is traversed as the stock history for each entry is downloaded from Yahoo's stock database. The data is saved to a 2-dimensional matrix using the 'DataFrame' attribute from the 'pandas' library. This automatically creates a formatted table including the stock price at each date in time between the bounds entered by the user. Using this data, the plot is then created and printed to the output of the program by simply using the plot() function from 'matplotlib.' The FindPriceAtTime class is designed to retrieve the closing price of a specific stock on a given date using data from Yahoo Finance. When an instance of the class is created, it takes a stock ticker symbol as input and stores it in uppercase to ensure consistency with Yahoo Finance's formatting. The class contains a method called get_price_at, which accepts a date in the format "YYYY-MM-DD". It first checks that the date is correctly formatted, then creates a date range using the given date and the following day (since Yahoo Finance returns stock data for date ranges, not just single days). It downloads the historical data for that period and checks if any data was returned. If available, it extracts and returns the closing price for the specified date, rounded to two decimal places. If the data is missing or the date format is incorrect, the method prints an appropriate error message and returns None.

[1]Python Software Foundation, "Datetime — Basic Date and Time Types — Python 3.7.2 Documentation," *Python.org*. https://docs.python.org/3/library/datetime.html
[2]"matplotlib.pyplot.plot — Matplotlib 3.4.1 documentation," *matplotlib.org*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html
[3]R. Aroussi, "yfinance: Yahoo! Finance market data downloader," *PyPI*, 2023. https://pypi.org/project/yfinance/

The system runs fairly autonomously. All the user need do is respond properly to the command prompts given by the program. Specific errors are returned if the user improperly enters dates and the names of certain stocks. The running of the class 'FindPriceAtTime' is also completely optional. The user is prompted with a choice on whether or not they need to find the value of a stock on a certain day. If they enter 'Y,' the program will run. If they enter 'N,' the code will break.

The program uses three special libraries in order to run. The first being the Yahoo Finance python library 'yfinance[3].' The second one is the time library 'datetime[1],' used to access specific dates. And finally the MATLAB plot library 'matplotlib[2].'

This project gave me a lot of insight on how far python extends outside of what we learned in class. For example, just the use of the three libraries discussed earlier in the report are so powerful in the way they can be used to create a multitude of programs. The python based API's I researched before starting my program were also super advanced. Due to the fact that I spent most of the two weeks which were allocated to complete this project studying for other final exams, I wasn't able to spend as much time as I would have liked completing it. If I had spent more time on the project, I would've liked to create an animated plot, capable of showing real-time stock prices as opposed to just historical stock data. I did try doing this, but it was fairly complicated so I decided to stick with what I had already produced. To future EECE 2140 students, I would really suggest choosing a topic you are passionate about and would consider using yourself for your final project. Due to the fact that I genuinely enjoyed what I was creating, I did put a lot more effort and research into it as opposed to the amount of effort I would've put into a physics lab report.

The outputs of my program are shown below. The following images show the stock trend graph for Apple (AAPL), Microsoft (MSFT), and Nvidia (NVDA). The stock price of Apple on 04/22/2025 is also given.

[1]Python Software Foundation, "Datetime — Basic Date and Time Types — Python 3.7.2 Documentation," *Python.org*. https://docs.python.org/3/library/datetime.html
[2]"matplotlib.pyplot.plot — Matplotlib 3.4.1 documentation," *matplotlib.org*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html
[3]R. Aroussi, "yfinance: Yahoo! Finance market data downloader," *PyPI*, 2023. https://pypi.org/project/yfinance/

Date	AAPL	MSFT	NVDA
2024-04-24	168.225662	405.960632	79.650826
2024-04-25	169.091583	396.016571	82.604866
2024-04-26	168.504333	403.241394	87.706184
2024-04-29	172.684616	399.202240	87.728172
2024-04-30	169.529495	386.380096	86.373619
...	...	...	...
2025-04-15	202.139999	385.730011	112.199997
2025-04-16	194.270004	371.609985	104.489998
2025-04-17	196.979996	367.779999	101.489998
2025-04-21	193.160004	359.119995	96.910004
2025-04-22	199.740005	366.820007	98.889999

[249 rows x 3 columns]

Figure 1: Stock DataFrame

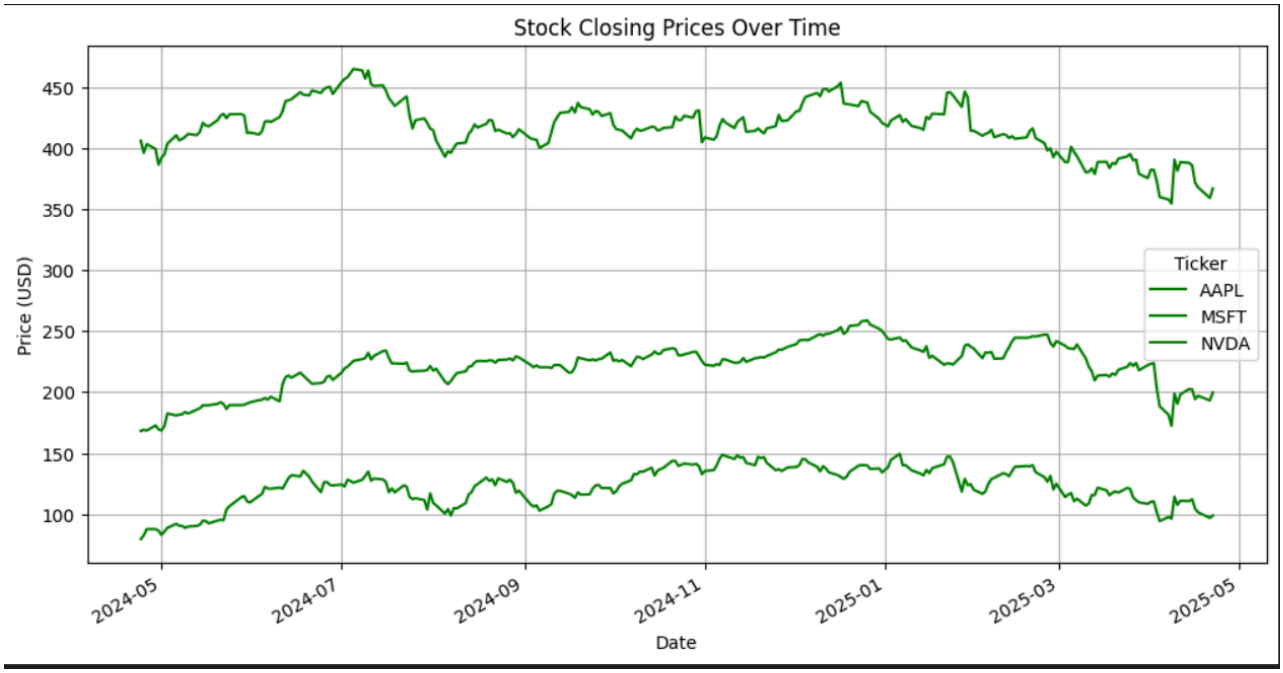

Figure 2: Stock Closing Prices Over Time

[1]Python Software Foundation, “Datetime — Basic Date and Time Types — Python 3.7.2 Documentation,” *Python.org*. https://docs.python.org/3/library/datetime.html
[2]“matplotlib.pyplot.plot — Matplotlib 3.4.1 documentation,” *matplotlib.org*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html
[3]R. Aroussi, “yfinance: Yahoo! Finance market data downloader,” *PyPI*, 2023. https://pypi.org/project/yfinance/

```
Closing price for AAPL on 2025-04-22 was: $Ticker
AAPL    199.74
```

Figure 3: Output of FindPriceAtTime for Apple on 04/22/2025

[1]Python Software Foundation, “Datetime — Basic Date and Time Types — Python 3.7.2 Documentation,” *Python.org*. https://docs.python.org/3/library/datetime.html

[2]“matplotlib.pyplot.plot — Matplotlib 3.4.1 documentation,” *matplotlib.org*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

[3]R. Aroussi, “yfinance: Yahoo! Finance market data downloader,” *PyPI*, 2023. https://pypi.org/project/yfinance/